Scalable Recommendation with Spark ALS

Amazon Reviews 2023: ID2221 Project Report

Group 0: Riccardo Fragale, Tommaso Tron, Edoardo F.A. de Cal

Implementation and dataset

We built a reproducible batch recommendation pipeline using PySpark/Spark SQL, Spark MLlib explicit-feedback ALS, Parquet, MongoDB, and Streamlit. Separate stages ingest data, sample users, construct temporal splits, train and evaluate models, benchmark scaling, and publish recommendations. Some run manifests preserve configuration, input checksums, seeds, code identity, and artifact lineage. All completed stages are protected against overwriting and recommendation batches can resume after interruption.

The dataset is the McAuley Lab's *Amazon Reviews 2023*, restricted to the category *Movies and TV*[1] as otherwise it would be too long and complex for our machines to compute. The raw category contains 17.3M reviews. We retain user ID, item ID (`parent_asin`), rating (1–5), and timestamp, plus a separate metadata projection for item titles. Keeping the latest review per user/item pair removes 170K duplicates, leaving 17.15M interactions, 6.5M users, and 750K items. Duplicates with equal timestamp retain the highest rating. Typed, validated interactions are stored in 27 yearly Parquet partitions (1997 to 2023; 756 MB). Unix timestamps are converted in UTC. Malformed records, missing IDs, out-of-range ratings, and implausible timestamps are counted by rejection reason in case they are found. Decompressed JSONL permits parallel input reads, whereas gzip input is not splittable. Metadata are deduplicated to one row per item so title joins cannot multiply recommendations.

Methodology

Development uses a deterministic, seeded 20% user sample, preserving each selected user's full cleaned history. A global chronological split targets 80/10/10 by interaction count, keeping identical timestamps together. Iterative training-only 5-core filtering removes users and items with fewer than five interactions until convergence. Held-out data are projected onto the retained training vocabulary; dense, deterministic integer mappings are persisted for ALS. Sampling uses a stable hash of the original user ID and seed 2221(selected randomly), making cohorts reproducible and benchmark fractions nested. Split boundaries are selected from cumulative timestamp-group counts, so actual proportions may differ slightly from their targets. Filtering is never fitted on validation or test labels. For the original full-category training window, 5-core filtering retains 5.9M ratings (43%).

We compare four systems: interaction-count popularity; a regularized bias-only model; plain explicit ALS; and bias+ALS, which fits signed rating residuals:

$$\widehat{r}_{ui} = \mu + b_u + b_i + p_u^{\mathsf{T}} q_i.$$

Biases are fitted by alternating user/item updates, with regularization 10, at most 10 passes, and tolerance 10^{-4}. ALS uses 10 iterations. Validation NDCG@10 selects among ranks $\{10, 25, 50\}$ and regularizations $\{0.03, 0.1, 0.3\}$ for each ALS variant. Selected settings are rank 50 with regularization 0.03 (ALS) and 0.1 (bias+ALS). Popularity windows of 30, 90, 180, 365 days and all history are compared on validation; the selected window is 180 days.

After settings are frozen, models are refitted on full-category train+validation history through 17 November 2020, recomputing the 5-core and mappings. This leaves 6.7 training ratings, 598,310 users, and 175,550 items. The later test window is untouched by fitting. Ranking uses exact full-catalog scores, excluding seen items, with deterministic ties. Ratings ≥ 4 define relevance; Precision@10 and binary NDCG@10 are macro-averaged over the same 44,612 eligible users. RMSE uses the same 162,922 training-known test ratings, with predictions clipped to $[1, 5]$. Popularity's rating predictor is a smoothed item mean, separate from its ranking counts. Cold-start exclusions and serving fallbacks are reported separately. Precision@10 divides the number of relevant recommendations by ten even for a short list; NDCG discounts hits by $1/\log_2(r+1)$ and normalizes by the ideal ordering of up to ten relevant items. Users without an eligible relevant test item are excluded and counted. Raw predictions remain available for diagnostics, while ranking scores are not clipped. User-level bootstrap intervals quantify ranking uncertainty, and a uniformly random unseen-item list supplies a reference.

Exact ALS ranking broadcasts the item-factor matrix and scores user blocks with NumPy inside Spark partitions, masking seen items before choosing the top items. For bias+ALS, item biases are included in the ranking score; adding user bias and the global mean restores full rating predictions. Unlike latent-only retrieval, this approach cannot omit a high-bias item merely because its residual score is low.

[1] Dataset and category downloads: `https://amazon-reviews-2023.github.io/`Citation: Hou et al., *Bridging Language and Items for Retrieval and Recommendation*, arXiv:2403.03952, 2024.

Recommendation quality

System	NDCG@10	Precision@10	Users with a hit	RMSE
Popularity (180 days)	0.00340	0.00093	415	1.263
Bias-only	0.00007	0.00004	17	**1.151**
Plain ALS	0.00015	0.00006	27	1.448
Bias+ALS	0.00003	0.00002	7	1.173

Recent popularity is the strongest ranker; its NDCG@10 bootstrap 95% interval is $[0.0030, 0.0038]$. Bias-only predicts ratings best, beating the training-global-mean reference (RMSE 1.355). Explicit ALS does not improve on recent popularity for ranking or on biases for rating prediction. Absolute ranking accuracy remains low; the sparse retained graph and substantial cold-start exclusions limit coverage. The random reference expects only 7.1 users with a top-10 hit, compared with 415 for recent popularity and 27 for ALS. Bias+ALS produces seven, approximately the random level. All-history popularity also produces only seven, indicating that recency matters in this temporal evaluation. ALS's training RMSE of 0.148 versus test RMSE of 1.448 indicates poor generalization for the validation-selected configuration. Selecting for NDCG does not necessarily optimize rating accuracy.

Scaling, serving, and limitations

The benchmark we made uses nested 25/50/100% user samples with shared temporal cutoffs and frozen model settings. Work times exclude Spark startup and verification counts; ETL is measured separately and MongoDB publication is excluded.

Work time (seconds)	25%	50%	100%
Temporal split and filtering	34.3	56.0	120.6
ALS training	22.6	38.7	72.3
Bias+ALS training	34.0	53.3	97.4
Exact ALS top-20	27.8	70.7	277.8
Exact bias+ALS top-20	27.6	74.5	335.2
Total, including other stages	185.6	349.9	972.6

Total work grows 5.24× for approximately 4× input. Exact ALS-based ranking accounts for 63% of work at full scale and grows with users times catalog size. Changing 5-core retention also changes the workload. These measurements demonstrate single-machine input scaling, not multi-machine speedup; there are no repetitions to estimate timing variability. Cluster deployment is unsupported by the current configuration validator. Item-factor broadcasting and collection of per-user evaluation metrics also impose memory limits. The split grows 3.51× for 4× input; ALS training grows 3.20× for 4.9× retained training rows. In contrast, the retained user–item catalog product grows 12.5×, explaining why exact ranking grows much faster than training. Event logs record shuffle, spill, garbage collection, and task skew to support bottleneck analysis. Each stage starts a new Spark process, and failed or timed-out stages remain visible in benchmark records.

The serving publication uses popularity with a 180 day window for every user, chosen because of its superior ranking results. All 6.5M users receive 20 unique unseen items: 600K known users and 5.9M fallback users. Thus the deployed ranking is not personalized. Bounded batches produce Parquet serving artifacts; idempotent bulk upserts write versioned MongoDB collections. An active pointer switches only after count and sampled-document validation. Streamlit looks up original user IDs and displays recommendations and saved experiment evidence. Missing titles are explicitly shown (using derived metadata labels or IDs). Fallback users comprise also 5.2M users removed by the 5-core run. Coverage audits check list length, uniqueness, rank order, finite scores, and exclusion of previously seen items. All published lists have length 20, but serving coverage must not be confused with the much smaller training-known cohort used for model-quality comparisons. Compared with the proposal, exact custom ranking replaces `recommendForUserSubset`, and custom Spark metric calculations replace MLlib evaluators to expose cohort, coverage, and short-list handling.

Running the code

Use Python 3.11, Java 17, and `uv`; Docker Compose is required for MongoDB. Install dependencies with `uv sync --frozen`. Configure the MongoDB credentials using `.env.example` and export `MONGODB_URI`. After running the pipeline and publishing recommendations, launch Streamlit with `make dashboard CONFIG=configs/real.toml` at `http://localhost:8501`.

Full instructions are provided in `README.md`, including dataset downloads, environment setup, pipeline commands, experiment reproduction, and MongoDB configuration.